

Making sense of MVP (Minimum Viable Product) — and why I prefer Earliest Testable/Usable/Lovable

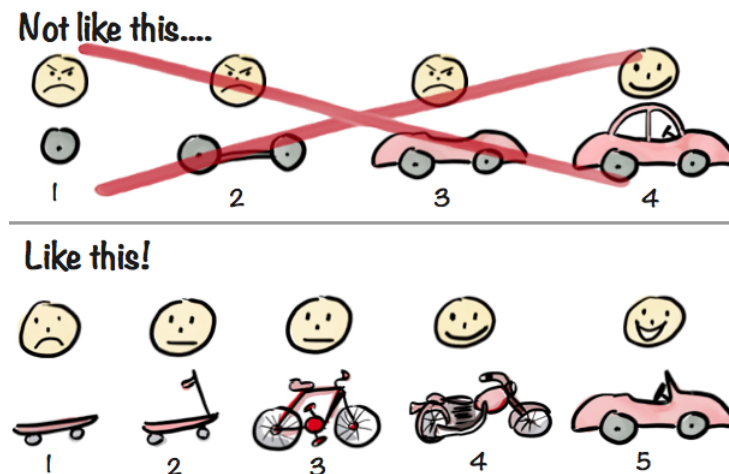


Posted on 2016-01-25 — 12:14

by [Henrik Kniberg](#)

([French translation](#), [Spanish translation](#), [Japanese translation](#))

A couple of years ago I drew this picture and started using it in various presentations about agile and lean development:



Since then the drawing has gone viral! Shows up all over the place, in articles and presentations, even in a book (Jeff Patton's "[User Story Mapping](#)" — an excellent read by the way). Many tell me the drawing really captures the essence of iterative & incremental development, lean startup, MVP (minimum viable product), and what not. However, some misinterpret it, which is quite natural when you take a picture out of it's original context. Some criticize it for oversimplifying things, which is true. The picture is a metaphor. It is not about actual car development, it is about product development in general, using a car as a metaphor.

Anyway, with all this buzz, I figured it's time to explain the thinking behind it.

First example — Not Like This

The top row illustrates a common misconception about iterative, incremental product development (a.k.a Agile).



Many projects fail badly because they do Big Bang delivery (build the thing until 100% done and deliver at the end). I've lost count of the number of failed projects I've seen because of this (scroll down for some examples). However, when Agile is presented as an alternative people sometimes balk at the idea of delivering an unfinished product — who wants *half* of a car?. Imagine this:

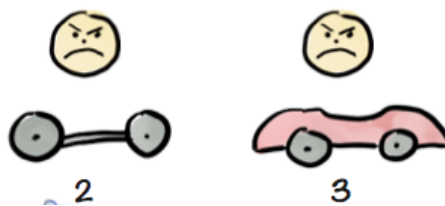
“Here sir, here’s our first iteration, a front tire. What do you think?”



Customer is like “Why the heck are you delivering a tire to me? I ordered a CAR! What am I supposed to do with this?”

(By the way, I’m using the term “customer” here as a generic term to represent people like product managers, product owners, and early adopter users).

With each delivery the product gets closer to done, but the customer is still angry because he can’t actually use the product. It’s still just a partial car.



And finally, when the product is done, the customer is like “Thanks! Finally! Why didn’t you just deliver this in the first place, and skip all the other useless deliveries?”.



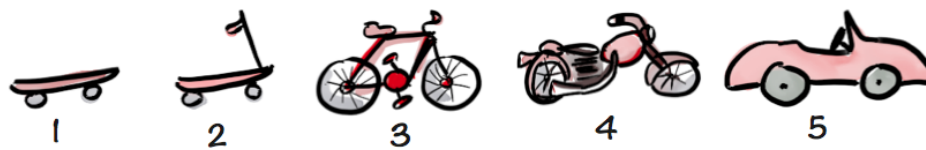
In this example he's happy with the final product, because it's what he ordered. In reality, that's usually not true. A lot of time has passed without any actual user testing, so the product is most likely riddled with design flaws based on incorrect assumptions about what people need. So that smiley face at the end is pretty idealistic.

Anyway, the first row represents "bastardized agile". Technically it might be incremental and iterative delivery, but the absence of an actual feedback loop makes it very risky — and definitely not agile.

Hence the "Not Like This" heading.

Second example — Like this!

Now for the second row.



Here we take a very different approach. We start with the same context — the customer ordered a car. But this time we don't just build a car. Instead we **focus on the underlying need the customer wants fulfilled**. Turns out that his underlying need is "I need to get from A to B faster", and Car is just one possible solution to that. Remember, car is just a metaphor, think any kind of customized product development situation.

So the team delivers the smallest thing they can think of that will get the customer testing things and giving us feedback. Some might call it an MVP (Minimum Viable Product), but I prefer to call it Earliest Testable Product (more on that further down).



Call it what you like (some even call their first release the "the skateboard version" of the product, based on this metaphor....).

The customer is unlikely to be happy with this. This is nowhere near the car he ordered. But that's OK! Here's the kicker — **we're not trying to make the customer happy at this point**. We might make a few early adopters happy (or pragmatists in pain), but our **main goal at this point is just to learn**. Ideally, the team explains this clearly to the customer in advance, so he isn't too disappointed.

However, as opposed to the front wheel in the first scenario, the skateboard is actually a usable product that helps the customer get from A to B. Not great, but a tiny bit better than nothing. So we tell the customer “don’t worry, the project is not finished, this was just the first of many iterations. **We’re still aiming to build a car, but in the meantime please try this and give us feedback**”. Think big, but deliver in small functionally viable increments.

*We might learn some really surprising things. Suppose the customer says he hates the skateboard, we ask why, and he says “I hate the color”. We’re like “uh.... the color? That’s all?”. And the customer says “Yeah, make it blue! Other than that, it’s fine!”. You just saved *alot* of money not building the car! Not likely, but who knows?*

The key question is “**What is the cheapest and fastest way we can start learning?**” Can we deliver something even earlier than a skateboard? How about a bus ticket?



Will this help solve the customer’s problem? Maybe, maybe not, but we’ll definitely learn something by putting this into the hands of real users. [Lean Startup](#) offers a great model based on listing your actual hypotheses about the users and then working systematically to validate or invalidate them.

You don’t need to test the product on all users, and you don’t need to build a product to test something. **Testing a prototype on even a single user will teach you more than nothing.**

But OK, back to the skateboard example.

After playing around with it in the office, the customer says “OK, kind of fun, and it does get me to the coffee machine faster. But it’s unstable. I fall off too easily”.

So the next iteration we try to solve that problem, or at least learn more about it.



Customer can now get around the office without falling off!

Happy? Not really, he still kind of wants that car. But **in the meantime he is actually using this product, and giving us feedback.** His biggest complaint is that it's hard to travel longer distances, like between buildings, due to the small wheels and lack of breaks. So, next release the product morphs into something like a bicycle.



Now the customer can zoom around campus. Yihaaa!

We learn some things along the way: The customer likes the feeling of fresh air on his face. The customer is on a campus, and transportation is mostly about getting around between buildings.

The bicycle may turn out to be a much better product than the car originally envisioned. In fact, while testing out this product we may learn that the paths are too narrow for a car anyway. We just saved the customer tons of time and money, and gave him a better product in less time!

*Now you may be thinking “but shouldn’t we already have known that. via up-front analysis of the customer’s context and needs?” Good point. But in most real-life product development scenarios I’ve seen, **no matter how much up-front analysis you do, you’re still surprised when you put the first real release into the hands of a real user**, and many of your assumptions turn out to be way off.*

So yes, do some up-front analysis, discover as much as you can before starting development. But don’t spend too much time on it and don’t trust the analysis too much – start prototyping and releasing instead, that’s when the real learning happens.

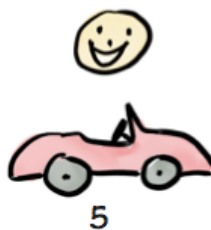
Anyway, back to the story. Perhaps the customer wants more. Sometimes he needs to travel to another city, and the bike ride is too slow and sweaty. So next iteration we add an engine.



This model is especially suitable for software, since software is, well, Soft. You can “morph” the product as you go, as opposed to hardware where you essentially have to rebuild every time. However, even in hardware projects there is a huge benefit to delivering prototypes to observe and learn from how the customer uses your product. It’s just that the iterations tend to be a bit longer (months instead of weeks). Even actual car companies like Toyota and Tesla do a lot of prototyping (sketches, 3d models, [full-scale clay models](#), etc) before developing a new car model.

So now what? Again, maybe the customer is happy with the motorcycle. We could end the project earlier than planned. Most products are riddled with complexity and features that nobody uses. The iterative approach is really a way of delivering less, or **finding the simplest and cheapest way to solve the customer’s problem**. Minimize the distance to Awesome. Very Zen.

Or, again, the customer could choose to continue, with or without modifications to the requirements. We may in fact end up with the exact same car as originally envisioned. However it is much more likely that we gain vital insights along the way and end up with something slightly different. Like this:



The customer is overjoyed! Why? Because we learned along the way that he appreciates fresh air in his face, so we ended up with a convertible. **He did get a car after all – but a better car than originally planned!**

So let’s take a step back.

What’s your skateboard?

The top scenario (delivering a front tire) sucks because we keep delivering stuff that the customer can’t use at all. If you know what you’re doing – your product has very little complexity and risk, perhaps you’ve built that type of thing hundreds of times before – then go ahead and just do big bang. Build the thing and deliver it when done.

However, most product development efforts I’ve seen are much too complex and risky for that, and the big bang approach all too often leads to huge expensive failures. So the key question is **What’s your skateboard?**

In product development, one of the first things you should do (after describing what problem you are trying to solve for whom) is to identify your skateboard-equivalent. **Think of the skateboard as a metaphor for the smallest thing you can put in the hands of real users, and get real feedback.** Or use “bus ticket” if that metaphor works better.

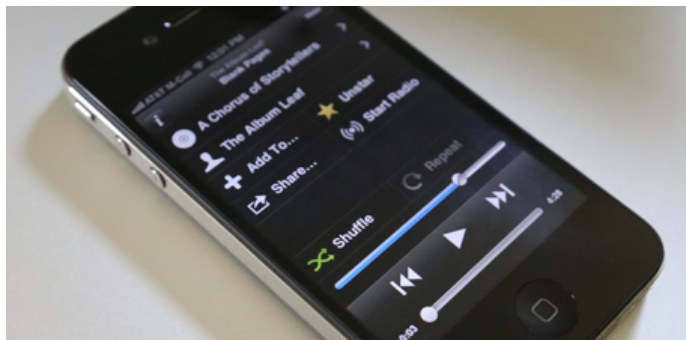
This will give you the vitally needed feedback loop, and will give both you and the customer control over the project — you can learn and make changes, instead of just following the plan and hoping for the best.

Let’s take at some real-life examples.

Example 1: Spotify music player

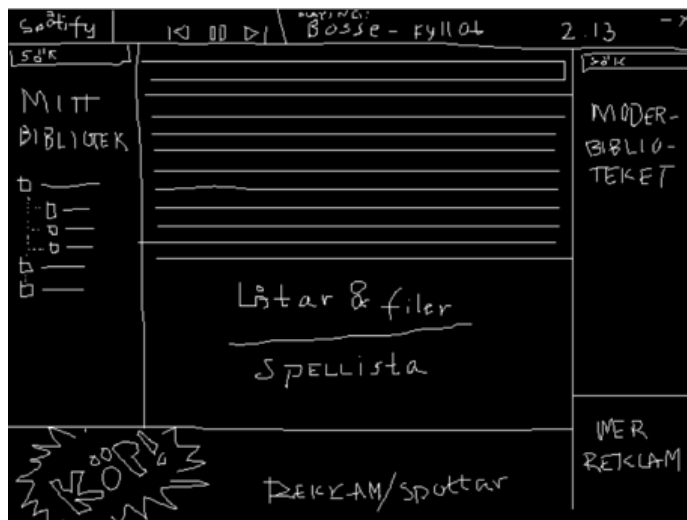
“With over 75 million users, it’s hard to remember a time without Spotify. But there was. A time when we were all mulling the aisles of Target for new CDs. A time in our lives where we all became thieves on Napster. A time when iTunes forced us to buy songs for \$2/piece. And then came Spotify.” — [Tech Crunch](#)

[Spotify](#) is a pretty fancy product now. But it didn’t start that way. I was lucky to be involved pretty early in this amazing journey (and still am).



As a startup in 2006, Spotify was founded on some key assumptions — that people are happy to stream (rather than own) music, that labels and artists are willing to let people do so legally, and that fast and stable streaming is technically feasible. Remember, this was 2006 when music streaming (like Real Player) was a pretty horrible experience, and pirate-copied music was pretty much the norm. The technical part of the challenge was: “Is it even possible to make a client that streams music instantly when you hit the Play button? Is it possible to get rid of that pesky ‘Buffering’ progress bar?”

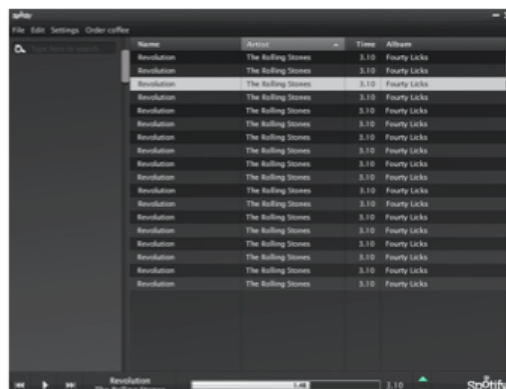
Starting small doesn’t mean you can’t think big. Here’s one of the early sketches of what they had in mind:



But instead of spending years building the whole product, and then finding out if the assumptions hold, the developers basically sat down and hacked up a technical prototype, put in whatever ripped music they had on their laptops, and started experimenting wildly to find ways to make playback fast and stable. The driving metric was “how many milliseconds does it take from when I press Play to when I hear the music?”. It should play pretty much instantly, and continue playing smoothly without any stuttering!

“We spent an insane amount of time focusing on latency, when no one cared, because we were hell bent on making it feel like you had all the world’s music on your hard drive. Obsessing over small details can sometimes make all the difference. That’s what I believe is the biggest misunderstanding about the minimum viable product concept. That is the V in the MVP.” -Daniel Ek, co-founder and CEO

Once they had something decent, they started testing on themselves, their family, and friends.



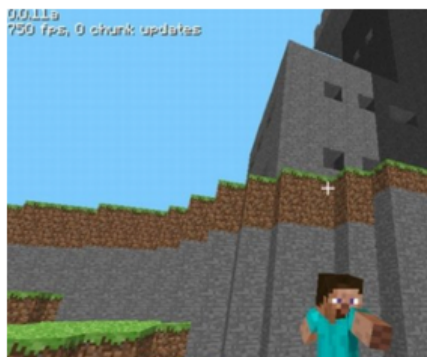
The initial version couldn’t be released to a wider audience, it was totally unpolished and had basically no features except the ability to find and play a few hard-coded songs, and there was no legal agreement or economic model. It was their skateboard.

But they **shamelessly put the skateboard in the hands of real users** — friends and family — and they quickly got the answers they needed. Yes, it was technically possible. And yes, people absolutely loved the product (or more like, what the product can become)! The **hypotheses were validated!** This running prototype helped convince music labels and investors and, well, the rest is history.

Example 2: Minecraft



Minecraft is one of the most successful games in the history of game development, especially if you take development cost into consideration. Minecraft is also one of the most extreme examples of the release-early-and-often mindset. The first public release was made after just 6 days of coding, by one guy! You couldn't do much in the first version — it was basically an ugly blocky 3d-landscape where you can dig up blocks and place them elsewhere to build crude structures.



That was the skateboard.

The users were super-engaged though (most developer-user communication happened via Twitter, pretty funny). Among the early users were me and my four kids. Over hundred releases were made during the first year. Game development is all about finding the fun (some game companies I've worked with use the term "Definition of Fun" instead of "Definition of Done"), and the best way to do that is by having real people actually play that game — in this cases thousands of real people who had actually paid to try the early access version and therefore had a personal incentive to help improve the game.

Gradually a small development team was formed around the game (mostly 2 guys actually), and the game became a smash hit all over the world. I don't think I've met any kid anywhere who doesn't play Minecraft. And last year the game (well, the company that was formed around the game) was sold to Microsoft for \$2.5 billion. Quite amazing.

Example 3: Big Government Project

Around 2010 the Swedish Police started a big initiative to enable police to spend more time in the field and less time at the station — PUST (Polisens Utrednings STöd). A fascinating project, I was involved as coach and wrote a book about what we did and what we learned (Lean from the Trenches).

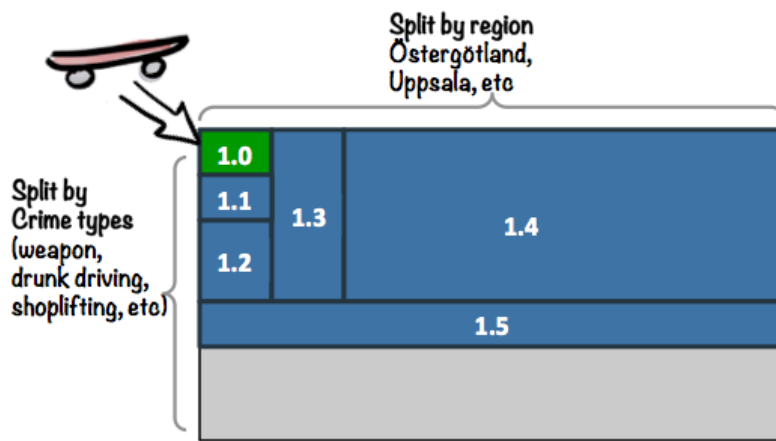


The idea was to put laptops in the cars, and customized software to give police access to the systems they need in real-time, for example while interrogating a suspect (this was the pre-tablet age).

They had tried to build similar systems in the past and failed miserably, mainly because of Big Bang thinking. They told me that one of their previous attempts took 7 years from inception to first release. SEVEN YEARS! By then of course everything had changed and the project was a total failure. So this time they wanted to do it differently.

The 60-person project (later referred to as “PUST Java”) succeeded surprisingly well, especially for being a big government project (it even came second in CIO Awards “Project of the Year”). One of the main success factors was that they **didn't try to build the whole thing at once** — they split the elephant along two dimensions:

- By Region. We don't need to release to ALL of Sweden at once, we could start by releasing to just one region.
- By Crime type. We don't need to support ALL crime types initially, we could start by just supporting 1-2 crime types.



The first version, 1.0, was their skateboard.

It was a small system, supporting just a couple of crime types, and it was field-tested on a handful of cops in Östergötland (a region in Sweden). Other crime types had to be dealt with the old way — drive to the station and do paperwork. **They knew they were guinea pigs, and that the product was nowhere near finished.** But they were happy to test it, because they knew the alternative. They had seen what kind of crappy systems come out of processes that lack early user feedback, and now they **finally had a chance to influence a system while it was being built!**

Their feedback was harsh and honest. Many of our assumptions flew out the window, and one of the big dilemmas was what to do with all the carefully crafted Use Case specifications that were getting less and less relevant as the real user feedback came in (this was an organization with a waterfall history and a habit of doing big upfront analysis).

Anyway, long story short, the **early feedback was channeled into product improvements** and, gradually, as the those cops in Östergötland started liking the product, we could add more crime types and spread it to more regions. By the time we got to the big release (1.4), with nationwide rollout and training of 12000 police, we weren't all that worried. **We had done so many releases, so much user testing, that we slept well on the night of the nationwide release.**

Unfortunately the victory was short-lived. A follow-up project (PUST Siebel) botched it and went back to waterfall-thinking, probably due to old habit. 2 years of analysis and testing without any releases or user-testing, followed by a big-bang release of the “next generation” of the product to all 12,000 police at once. It was an absolute disaster, and after half a year of hemorrhaging they shut the whole thing down. The development cost was about €20 million, but Internal studies estimate that the cost to Swedish society (because the police were handicapped by the horrible system) was on the order of €1 Billion!

Pretty expensive way to learn!

Example 4: Lego

I'm currently working at [Lego](#), and I'm amazed by their ability to deliver new smash-hits, year after year without fail. I hear lots of interesting stories about how they do this, and the common theme is prototyping and early user testing! I often see groups of kids in the office, and designers collaborate with local kindergartens and schools and families to field-test the latest product ideas.

Here's a recent example — [Nexo Knights](#) (released Jan 2016):



When they first started exploring this concept, they did paper prototypes and brought to small kids. The kids' first reaction was "hey, who are the bad guys? I can't see who's good and who's bad!". Oops. So the designers kept iterating and testing until they found a design that worked with the kids. I bet even you can see who's good and who's evil in the picture above...

Not sure exactly where the skateboard is in this story, but you get the idea — **early feedback from real users!** Don't just design the product and build the whole thing. Imagine if they had built the product based on their original design assumptions, and learned about the problem *after* delivering thousands of boxes to stores all over the world!

Lego also has it's share of hard-earned failures. One example is [Lego Universe](#), a massively multiplayer online Lego world. Sounds fun huh? Problem is, they got overambitious and ended up trying to build the whole thing to perfection before releasing to the world.



About 250 people worked for 4-5 years (because of constant scope creep due to perfectionism), and when the release finally came the reception was... lukewarm. The finished game was beautiful, but not as fun as expected, so the product was shut down after 2 years.

There was no skateboard!

Why not? Because skateboards aren't Awesome (at least not if you're expecting a car), and Lego's culture is all about delivering Awesome experiences! If you work at Lego HQ in Billund, Denmark you walk past this huge mural every day:



Translates roughly to “Only the best is good enough”. It has been Lego's guiding principle ever since the company started 80+ years ago, and has helped them become one of the most successful companies in the world. But in this case the principle was misapplied. **The hunt for perfection delayed vital feedback**, which meant mistaken assumptions about what the users like and don't like. The exact opposite of Minecraft.

Interestingly enough the Lego Universe teams were actually using Scrum and iterating heavily — just like the Minecraft guys did. But the releases were only internal. So there was most likely a skateboard, and a bicycle, and so on, but those products never reached real users. That's not how Scrum is intended to be used.

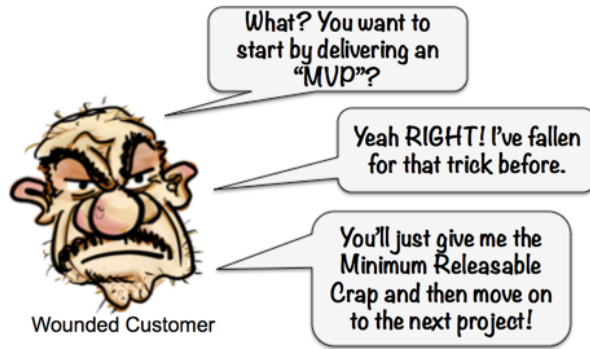
It was an expensive failure, but Lego learned from it and they are constantly getting better at early testing and user feedback.

Improving on “MVP”

And that (deep breath...) brings me to the topic of MVP — Minimum Viable Product.

The underlying idea is great, but the term itself causes a lot of confusion and angst. I've met many customers that are like “No Way do I want an MVP delivery — that's the last delivery I'll get!” All too often teams deliver the so-

called Minimum Viable Product, and then quickly get whisked away to the next project, leaving the customer with a buggy, unfinished product. For some customers, **MVP = MRC (Minimum Releasable Crap)**.



I know, I know, this comes down to bad management rather than the term MVP, but still... the term invites misunderstanding. "Minimum" and "Viable" mean different things to different people, and that causes problems.

So here's an alternative.

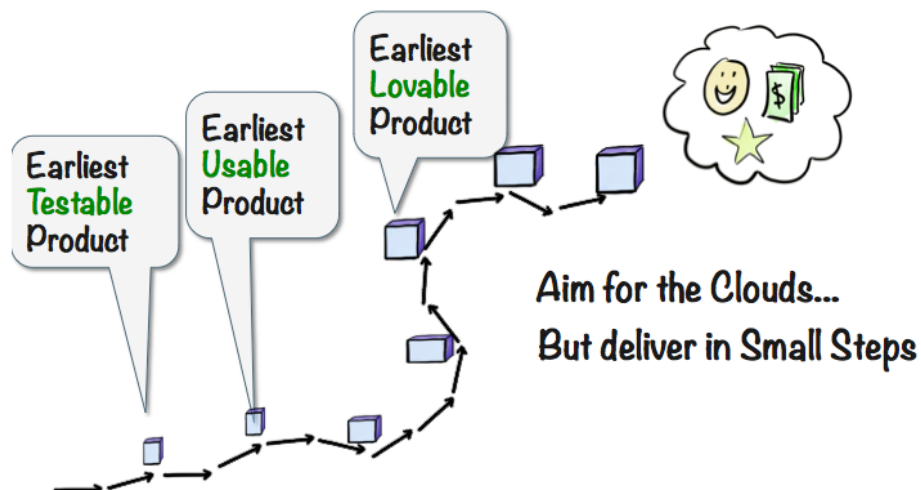
First of all, replace the word "Minimum" with "Early". The whole idea behind releasing an MVP is to get early feedback — by delivering a minimum product rather than a complete product, we can get the feedback earlier.

Few customers want "minimum" but most customers want "early"! So that's our first change:

Minimum => Earliest

Next, remove the word "Viable" since it's too vague. Your "viable" is my "horrible". Some people think Viable means "something I can test and get feedback from", others think it means "something the customer can actually use". So let's be more explicit and split it into three different things:

Minimum viable => Earliest testable/usable/lovable



Earliest Testable Product is the skateboard or bus ticket — the **first release that customers can actually do something with**. Might not solve their problem, but it generates at least some kind of feedback. We make it very clear that learning is the main purpose of this release, and that any actual customer value will be a bonus.



Earliest Usable Product is perhaps the bicycle. The **first release that early adopters will actually use, willingly**. It is far from done, and it might not be very likeable. But it does put your customers in a better position than before.



Earliest Lovable Product is perhaps the motorcycle. The **first release that customers will love, tell their friends about, and be willing to pay for**. There's still lots to improve, and we may still end up with a convertible, or a plane, or something else. But we've reached the point where we have a truly marketable product.



*I considered adding an even earlier step "**Earliest Feedbackable Product**", which is basically the paper prototype or equivalent that you use to get your first feedback from the customer. But four steps seems too many, and the word Feedbackable..... ugh. But nevertheless, that is also an important step. Some would call a paper prototype the Earliest Testable Product, but I guess that comes down to how you define Testable. Check out [Martin's MVP Guide](#) to learn more — he's got plenty of super-concrete examples of how to get early feedback with minimum investment.*

Of course people can still misinterpret Earliest Testable/Usable/Lovable, but it's at least one step more explicit than the nebulous Minimum Viable Product.

Takeaway points

OK time to wrap it up. Never thought it would get this long, thanks for sticking around! Key takeaways:

- **Avoid Big Bang** delivery for complex, innovative product development. Do it iteratively and incrementally. You knew that already. But are you

actually doing it?

- **Start by identifying your skateboard** — the earliest testable product. Aim for the clouds, but swallow your pride and start by delivering the skateboard.
- **Avoid the term MVP.** Be more explicit about what you're actually talking about. Earliest testable/usable/lovable is just one example, use whatever terms are least confusing to your stakeholders..

And remember — the skateboard/car drawing is just a metaphor. Don't take it too literally :o)

PS — here's a fun story about how my kids and I used these principles to win a [Robot Sumo competition](#) :o)

Thanks Mary Poppendieck, Jeff Patton, Alistair Cockburn, Anders Haugeto, Sophia, colleagues from Crisp, Spotify and Lego, and everyone else who gave tons of useful feedback.



Posted on 2016-01-25 — 12:14

by [Henrik Kniberg](#)

[agile](#)

[lean startup](#)

[mvp](#)

[Subscribe to Henrik's RSS feed](#)

216 responses on “Making sense of MVP (Minimum Viable Product) — and why I prefer Earliest Testable/Usable/Lovable”



Stan

2016-01-25 at 16:32

Hey Henrik,

Firstly, thanks for this article and explanation. You made a great job on this and I would like to thank you.

Secondly, I am from those who critique your metaphor and here is why:

1) There is another image with a completely wrong sense. You can find it here: [https://www.linkedin.com/nhome/updates?](https://www.linkedin.com/nhome/updates?topic=6093258891330609152)

[topic=6093258891330609152](https://www.linkedin.com/nhome/updates?topic=6093258891330609152) (also, there you can find some comments, why personally I don't like). Its a shame, that the version I shared in this link is more popular than the original image (yours). As you can compare both images, there is a big differences in user's emotions in